Does One Integer Still Price Work and Memory in Parallel? A Preregistered Experiment on Interaction Combinators

true

2026-08-24

Contents

What this document is

Σ-GLYPH Book I proves that a single unsigned integer prices both the work an evaluation performs and the peak memory it materialises, at every configuration of a **sequential** machine. This report asks whether that carries over to a setting where reduction is local and confluent and no order is fixed, and answers it by measuring one small reducer for Lafont's interaction combinators.

It is short, and it is deliberately narrow. It measures nets, not programs; it measures one reducer, not any existing runtime; and half of its question is settled by arithmetic rather than by experiment, which the preregistration says in advance so that the arithmetic cannot later be presented as a discovery.

Why the two parts are printed together

The point of this record is as much the method as the result. What follows is:

- **Part I — the preregistration**, exactly as committed at `d3eea63`, *before* the reducer existed. It fixes three hypotheses, names what would make the experiment worthless, and states that the result would be written whichever way it came out.
- **Part II — the result**, which was written after the measurement and then corrected three times in review. Its corrections section names what earlier versions claimed and why those statements were wrong, rather than repairing them silently.

Neither part has been edited to agree with the other. Where the preregistration's wording turned out to be ambiguous — hypothesis H2 says "a factor that grows with net size" without fixing whether the factor is absolute or relative — the result says so and the hypothesis text is left alone, because editing a hypothesis after the numbers is the one thing a preregistration exists to prevent.

Provenance

repository	https://github.com/s0fractal/sigma-glyph
preregistration	`experiments/EXP-004-parallel-bound-preregistration.md`
result, code, receipt	`experiments/exp-004/`
corpus	29 nets, pinned by exact structure in `fixtures.json`
receipt	`results.json`, byte-identical across two local replays and a Linux CI runner
controls	nine, each broken in turn by `selftest.py` and required to fail for its own reason

The deposited snapshot of the repository accompanies this document. On the archived commit, the experiment's own workflow re-derives `results.json` and fails if it differs from the committed one.

What this record does not claim

It is not peer reviewed. A DOI is a permanent address and a frozen artifact; it is not a venue and not an endorsement.

It is not evidence about HVM or any other runtime: none was measured, read or run. Its statement about interaction counts is a statement about counting.

Its schedules are greedy rather than optimal, so every difference it reports between schedules is a lower bound. Its transient-memory figures depend on an allocation discipline that was chosen and declared, not

derived from the rewrite rules. Its observation that peaks stayed within 1.5× across schedules on normalising nets is a property of 29 nets and not a bound.

Author contribution and model use

The reducer, the corpus, the harness, the controls and this prose were written by an AI model (Anthropic Claude, `claude-opus-5`) working under the author's direction and authority. Three rounds of adversarial review were conducted by separate fresh-context model sessions — OpenAI Codex and a second Claude session — prompted by the author.

That review found nine defects, **six of them in the controls rather than in the measurement**: a schedule capped on rounds instead of interactions, a receipt carrying one schedule's count for four, a transient formula that did not match its own description, an allocation profile checked only through its difference so that `5` and `3` passed where the reducer does `4` and `2`, a batch schedule that truncated the round it was evidence about, and a reserved-then-released figure assembled from maxima of different rounds. None of the six was anticipated by the session that wrote the controls.

This is not independent audit. All reviewers were language-model sessions prompted by the same author, and the finding above is the reason the review happened at all rather than a reason to trust the artifact more.

Part I — The preregistration

Committed at `d3eea63`, before any reducer or measurement existed. Reproduced here unedited.

EXP-004 — does the one-integer bound survive parallel interaction nets?

Preregistration. No measurement has been taken. Non-normative: nothing here changes Book I or any released contract.

The question, stated so it can fail

Book I proves `size ≤ spent + 1` at every configuration a run passes through, and it rests on one per-step lemma: a fired action grows the term by at most `cost − 1` (`size_step` in `EvalMachine.lean`). The schedule is fixed — leftmost outermost, one action at a time.

Interaction nets make the opposite trade: rewriting is local and strongly confluent, so independent reductions proceed in any order and still meet. The question is whether a single integer can still price work *and* peak memory when no order is fixed.

What is already decided on paper, and should not be measured

Every interaction-combinator rule has a fixed net-size delta: annihilation removes two nodes, commutation replaces two with four, erasure against a binary agent leaves two. So `Δsize ≤ +2` per interaction, unconditionally, and

```
peak ≤ initial + 2 × interactions
```

follows by the same telescoping Book I uses. Publishing that as a finding would be dressing arithmetic as a result. **The form of the invariant transfers, and this experiment does not claim credit for it.**

What is genuinely open, and is what gets measured

Confluence promises the same *normal form* under any schedule. It promises nothing about the *peak*. So:

H1 — the peak is schedule-dependent. Two schedules reducing the same net to the same normal form reach different maximum sizes.

H2 — the gap is not marginal. Over the sampled nets, the worst schedule's peak exceeds the best schedule's by a factor that grows with net size rather than staying within a small constant.

H3 — the interaction count is schedule-invariant, or nearly so, for interaction combinators.

If H1 is false, one integer prices both exactly as it does in Book I, and the result is stronger than expected. If H1 is true, the honest statement is that a single budget can still *bound* both, but only against the worst schedule: the number you prepay stops predicting the memory you actually need, which is a weaker guarantee than the sequential machine gives.

Protocol

- implement Lafont's interaction combinators (γ, δ, ε) with the three rule families, in plain Python, no dependencies;
- two schedulers over the identical net: **sequential** (one redex at a time, a fixed deterministic choice) and **maximal-parallel** (every active pair reduced in one step, which is where confluence is doing the work);
- corpus: nets generated from a fixed seed list, plus hand-built nets that duplicate a shared structure, because duplication is where the peak lives;

- record per net: interactions, peak size, final size, and whether both schedules reach the same normal form;
- **negative control**: a net whose normal form differs between schedules would mean the implementation is wrong, not that confluence fails; the run fails rather than reports.

What would make this worthless

Choosing nets after seeing the numbers. The corpus and the seeds are fixed in the first commit of the implementation, before any measurement is recorded, and the result is written whichever way it comes out.

What this cannot say

It cannot say anything about HVM's internals, which are not measured here, and it cannot say Book I's theorem is wrong or right in a setting Book I does not claim. It measures one property of one small reducer, and the transfer question is about the *shape* of the invariant, not about anyone's implementation.

Date	Change	Result already known?
2026-08-23	initial preregistration	no
2026-08-23	result recorded in exp-004/RESULT.md; nothing above was edited	—
2026-08-23	four review defects corrected in the result; nothing above was edited	yes
2026-08-24	five further defects corrected; corpus pinned by exact structure, not regenerated; nothing above was edited	yes
2026-08-24	noted, not edited: H2 says "a factor that grows with net size" without fixing whether the factor is absolute or relative, so it can be answered both ways	yes

Part II — The result

Written after the measurement, corrected three times in review. Reproduced here unedited.

EXP-004 — result

Judged against EXP-004-parallel-bound-preregistration.md, written before the reducer existed. The corpus was committed at `3040b57`, before `measure.py` was written. No net was added or removed after a number was known.

reducer	Lafont's interaction combinators, `G`/`D<label>`/`E`, labelled duplicators as in HVM
corpus	29 nets in four families, pinned by digest in `fixtures.json`, 21 of which normalise
schedules	sequential, shrink-first, grow-first, maximal-parallel
budget	200,000 **interactions** for every schedule, or 40,000 agents
host	Python 3.14.7; corpus digest `9d3b8844ce0b3a66`, each net pinned by exact structure
gate	nine controls; `selftest.py` in CI on every push, full replay path-filtered

Corrections

Nine defects were found in two rounds of review after the first result was written. The result survives all nine; two of its statements did not, and one control turned out to be measuring nothing. They are named here rather than quietly repaired.

First round

1. **"Equal work, 200,000 interactions" was false.** The receipt recorded only the sequential schedule's interaction count and applied it to all four, and the parallel schedule was capped on *rounds* rather than interactions. The real counts on `random-3-12` were 200,000 / 200,000 / 19,995 / 1,199,988. The comparison has been re-run at genuinely equal work and the conclusion holds — see below — but the original receipt did not support it.
2. **"The enforcement does not transfer" was too strong.** A round-granular machine can prepay without arbitration. The corrected claim is narrower and sharper; it is now the main architectural finding.
3. **The transient figure did not match its own description.** It was computed as `size + 2 × growing`, which is "all growth before all shrinking", not "allocate everything before freeing anything". An explicit allocation model is now defined, and a control fails if it misstates any rule.
4. **The normal-form signature was not reproducible.** It used Python's `hash`, which is seeded per process: stable within a run, so the comparisons between schedules were sound, and different on the next run, so no result could be checked from its own record. Found by the new corpus pin on its first execution.

Second round

5. **The experiment was not in CI.** Every check on the pull request was green while no workflow ran `measure.py` or `selftest.py`. A green result about an artifact nobody executes is the exact shape of defect this repository keeps finding, and it had been reintroduced by the document reporting on it.
6. **The allocation control was still half empty.** It compared only `ALLOCATES − FREES`, so declaring `5` and `3` passed while the reducer does `4` and `2` — the same net change, a different widest point, which is the only thing the figure is for. The reducer now counts agents created and destroyed, and every interaction is compared against **both** numbers.

7. **Reserved-then-released spanned different rounds.** It was `max(envelope) − max(kept)` over the whole run. It is now per round, and it has an exact identity that makes it checkable — see below.
8. **The batch schedule truncated its last round.** Choosing part of a round *is* the arbitration that the architectural conclusion says round granularity avoids, so the schedule was contradicting the claim it was evidence for. There are now two: `parallel-round` runs a round or refuses it, and a prefix variant is used only where every schedule must stop at exactly the same count.
9. **The corpus pin used the non-injective signature, and `results.json` was written even when controls failed.** The pin is now the net's exact structure — symbols, every port's wire root, the interface — hashed; the receipt is written only by `--record`, and only after every control passes.

The short version

The bound transfers. Parallelism forces an explicit choice of refusal granularity — per redex, per prefix, or per whole round — and Book I has only the first.

What was settled on paper, and stayed settled

Every rule changes the agent count by at most +2: commutation replaces two agents with four, annihilation removes two, erasure against a binary agent replaces two with two. So

```
peak ≤ initial + 2 × interactions
```

and priced at 3 ATP per interaction this is exactly Book I's per-step shape, `Δsize ≤ cost − 1`, telescoping to `size ≤ spent + initial`. The preregistration said in advance that publishing this as a finding would be dressing arithmetic as a result, and it is not published as one. It is *checked* on every row instead, and never failed.

There is a structural reason it holds so easily. **A duplicator can never copy an active pair:** both principal ports in an active pair are occupied by each other, so nothing can reach them to copy them. Pending work cannot be duplicated. That is precisely what β-substitution lacks — a β-redex sits inside a term and is copied along with it, so the same work can be duplicated before it is done — and it is why a single work-and-memory budget is more at home here than in the setting Book I proves it for.

H1 — confirmed. The peak is schedule-dependent

13 of the 21 normalising nets reached different peaks under different schedules. The 8 that did not are the two control families plus one net already in normal form: `dup-tree` has no shrinking interaction, so every schedule *must* agree, and did. A spread there would have meant the harness was inventing one.

H2 — half right, and the half that is wrong matters

As an absolute count the gap grows without limit: `race-3-4` spreads by 8 agents, `race-7-256` by 254, linearly in the size of the net. As a **ratio** it does not: across every net that normalised, the worst schedule's peak stayed within **1.5×** the best. H2 predicted the gap would not stay within a small constant. On normalising nets, proportionally, it does.

Two caveats that the hypothesis's own wording invites. **1.5× is a property of this corpus, not a proved bound** — 29 nets up to a few hundred agents, and nothing here rules out a family whose ratio grows. And H2 said "a factor that grows with net size" without saying whether the factor was absolute or relative, which is why it can be answered both ways at once. A preregistration should fix the metric of a gap as an expression, not as the word *factor*; that is a lesson about writing hypotheses, and the text above has not been edited to hide it.

The finding neither hypothesis anticipated

The spread is exactly the reordering of a fixed multiset, and it is computable in advance.

Interaction nets are strongly confluent, so computation is unique up to trivial commutations of independent steps: every path to a normal form performs the same interactions, and only their order may differ.[1] The reachable peaks are therefore the prefix sums of one fixed sequence of `+2`/`0`/`−2` steps, and no two schedules can differ by more than

```
2 × min(growing interactions, shrinking interactions)
```

That is a prediction, not an observation, and it holds on all 21 normalising nets — reached **exactly** on 19. The two that fall short (`random-1-48`, `random-13-48`) do so because the greedy schedules cannot reach the extreme order, not because the bound is loose.

So the one integer survives the move to a confluent parallel setting, and the precision it loses is not unknown: it is bounded by a quantity computed from the same accounting that produced the budget.

Where it breaks: work that never finishes

On nets that do not normalise the multiset is no longer fixed, and the schedule decides whether memory is bounded at all.

`random-3-12` — twelve agents — with every schedule stopped at exactly the same **19,995 interactions**, which is where the greediest schedule hit the size cap:

schedule	peak agents at 19,995 interactions
shrink-first	14
sequential	16
maximal-parallel	20
grow-first	≥ 40,002 (size cap)

A factor of **2,857** on the same net doing the same amount of work. The bound `size ≤ spent + initial` is still true here and still useless: it tracks what was spent, and what was spent is a property of the schedule rather than of the computation. For a terminating computation that distinction collapses, which is why Book I never has to make it.

The architectural finding: parallelism chooses the granularity of refusal

Book I does not merely *bound* memory. It **refuses**: an action that cannot be afforded does not happen, and the check precedes the action. Applied per redex, that discipline needs a total order — with `k` pairs firing at once, deciding *which subset* to run when the budget covers only part of the round is arbitration, and arbitration is the serialisation interaction nets exist to avoid.

But a round-granular machine does not need it. For a maximal-parallel round: count `k`, reserve `3k` ATP and the round's allocation envelope, then run the whole round or refuse the whole round. No order among the `k` is required — and the schedule measured here really does refuse whole rounds, which a control enforces, because a schedule that quietly took a prefix would be doing the very arbitration this paragraph claims to avoid. Two rounds were refused for want of budget across the corpus, the largest of eight interactions.

So the correct statement is not that enforcement fails to transfer, but that

the bound transfers unchanged; Book I's per-redex partial-progress enforcement does not.

The price of that trade is measurable, and it has an exact form. Under the allocation profile — every rule builds its entire right-hand side before any agent of its left-hand side is freed, so a commutation holds six agents at its widest and an erasure four, both checked against the reducer's own counters on every interaction — the memory a round reserves and hands straight back is

```
envelope − what the round keeps = exactly the agents that round destroys
```

which is an identity, not an estimate, and a control compares the arithmetic against the reducer's free counter to keep it one. That quantity is nonzero on **28 of 29** nets. A sequential machine never holds more than **2** agents above its own peak; the round-granular machine held up to **20,102**. Refusing per round is simple, and it is not free; this experiment does not say which is cheaper.

Those figures belong to an ordering, not to the model. The round envelope assumes every allocation in a round precedes every free, which is one extreme. A machine that finishes each interaction before starting the next never exceeds the sequential transient, which is the other. Any real implementation sits somewhere between, so the pair — **2** and **20,102** — *brackets* the intra-round orderings rather than predicting the one a given machine will have. The per-rule profile is a declared choice too: `4 allocated before 2 freed` is checked against this reducer on every interaction, but it is a statement about how this reducer builds a right-hand side, not something the rewrite rules force. A different intra-step discipline is a different envelope, and this experiment measures one.

What this says about interaction counts as a cost model

An interaction count cannot price memory. It is invariant across schedules whose peaks differ by up to 1.5× when they terminate and by three orders of magnitude when they do not. A cost model reporting interactions alone reports the half of the pair that does not vary.

That is a claim about counting, **not about HVM**, whose implementation was not measured, read, or run here. Nothing in this document is evidence about any existing runtime.

What this cannot say

- The schedules are greedy, not optimal, so every spread reported is a **lower bound** on the true spread between the best and worst schedule.
- Memory is counted in agents. Counting wires would scale the same quantity, since an interaction changes the wire count by a bounded amount as well.
- The transient figures depend on an intra-step allocation discipline that was chosen and declared, not derived. They bound the orderings; they do not describe any particular machine.
- The **1.5×** ceiling on normalising nets is an observation about 29 nets. It is not a bound, and no argument here says one exists.
- Normal forms are compared by colour-refinement signature. Equal nets give equal signatures, so a mismatch is real; the converse does not hold. The honest statement is **no signature mismatch was detected**, not that the normal forms were proved isomorphic.
- No λ-calculus encoding was measured. Nothing here is about λ-terms, real programs, or sharing as an optimisation — only about nets.
- Eight nets did not normalise within the budget. Whether they normalise at all was not determined and is not claimed either way.
- The corpus is pinned by the exact structure of every starting net and the Python version recorded, because the nets come out of a seeded generator and the file alone does not say what was measured. The generator itself was **not** replaced with a version-independent one: that would have produced a different corpus after the numbers were known, which the preregistration forbids.

Controls

`measure.py` is check-only unless given `--record`, and writes the receipt only after every control has passed: a receipt beside a failure looks exactly like a receipt beside a success. It fails rather than reports if any schedule's observation is missing from the record, or is incomplete — an incomplete record is rejected before analysis, because a control that reads past a missing field crashes instead of reporting, which is a silent control; if a schedule stopping on the budget did not spend it in interactions; if the allocation model predicts a final size the net does not have; if any peak exceeds `initial + 2 × interactions`; if any starting net differs from its pinned digest; or, on a net that normalises, if the schedules disagree on the interaction count, the multiset of rules or the normal-form signature, or spread further than reordering permits.

`selftest.py` breaks each of those nine controls in turn and requires each to fail **for its own reason** — a control that cannot be made to fail is not a control, and a perturbation that breaks everything attributes nothing. Six of the perturbations reproduce defects this harness actually shipped: the parallel schedule capped on rounds, the record carrying one schedule's count for all four, a transient computed from a formula that did not match its description, a profile whose difference was right and whose widest point was wrong, a batch schedule truncating its last round, and reserved-then-released assembled from maxima of different rounds.

`selftest.py` runs in CI on every push. The full corpus replay runs in its own path-filtered workflow, check-only, and fails if the committed `results.json` differs from what the replay derives.

---

[1] Yves Lafont, *Interaction Combinators*, Information and Computation 137(1):69–101, 1997. doi:10.1006/inco.1997.2643

Date	Change	Result already known?
2026-08-23	initial preregistration	no
2026-08-23	result recorded; no hypothesis, threshold or net was edited	—
2026-08-23	four review defects corrected; conclusion narrowed, not widened	yes, and the corrections are listed above
2026-08-24	five further defects corrected, three of them in the controls themselves; experiment wired into CI	yes, and the corrections are listed above
2026-08-24	transient re-stated as a bracket over intra-step orderings; 1.5× marked as a sample property; H2's ambiguous wording named rather than repaired	yes